

# **Apprentissage par renforcement : cadre général et programmation dynamique**

Marin Ferecatu

Notes de cours

UE RCP211, Séance RL No. 1

CNAM Paris

Document rédigé en L<sup>A</sup>T<sub>E</sub>X

28 mars 2026

# Table des matières

|                                                                              |           |
|------------------------------------------------------------------------------|-----------|
| <b>Avant-propos</b>                                                          | <b>3</b>  |
| <b>Notations globales</b>                                                    | <b>4</b>  |
| <b>1 Motivations et caractéristiques de l'apprentissage par renforcement</b> | <b>5</b>  |
| 1.1 Pourquoi un cadre spécifique ?                                           | 5         |
| 1.2 L'agent                                                                  | 5         |
| 1.3 L'environnement                                                          | 6         |
| 1.4 Le feedback                                                              | 6         |
| 1.5 Récompense immédiate et récompense retardée                              | 6         |
| 1.6 Exemples de rewards                                                      | 7         |
| 1.7 Types d'environnement                                                    | 7         |
| 1.8 Schéma classique d'interaction en RL                                     | 8         |
| 1.9 Quels problèmes peut-on traiter ?                                        | 8         |
| 1.10 Exemples introductifs                                                   | 9         |
| 1.11 Quelques repères historiques                                            | 9         |
| 1.12 Applications pratiques actuelles du RL                                  | 10        |
| <b>2 Cadre théorique</b>                                                     | <b>11</b> |
| 2.1 État markovien                                                           | 11        |
| 2.2 Markov Reward Process                                                    | 11        |
| 2.3 Matrice de transition                                                    | 12        |
| 2.4 Politique et MDP                                                         | 12        |
| 2.5 Retour cumulé                                                            | 13        |
| 2.6 Fonction de valeur d'état $v_\pi$                                        | 13        |
| 2.7 Fonction de valeur d'état-action $q_\pi$                                 | 13        |
| 2.8 Valeurs optimales                                                        | 14        |
| 2.9 Valeurs optimales                                                        | 14        |
| 2.10 Équations de Bellman                                                    | 15        |
| 2.11 Exemple : Grid World                                                    | 19        |

|          |                                                       |           |
|----------|-------------------------------------------------------|-----------|
| <b>3</b> | <b>Model-based RL : programmation dynamique</b>       | <b>21</b> |
| 3.1      | Problème général . . . . .                            | 21        |
| 3.2      | Évaluation d'une politique . . . . .                  | 21        |
| 3.3      | Policy Iteration . . . . .                            | 24        |
| 3.3.1    | Quel problème essaie-t-on de résoudre ? . . . . .     | 24        |
| 3.3.2    | Étape mathématique d'évaluation . . . . .             | 24        |
| 3.3.3    | Étape mathématique d'amélioration . . . . .           | 25        |
| 3.3.4    | Intuition de la méthode . . . . .                     | 25        |
| 3.3.5    | Algorithme . . . . .                                  | 26        |
| 3.3.6    | Pourquoi est-on assuré de converger ? . . . . .       | 26        |
| 3.4      | Value Iteration . . . . .                             | 26        |
| 3.4.1    | Quel problème essaie-t-on de résoudre ? . . . . .     | 26        |
| 3.4.2    | Mise à jour mathématique . . . . .                    | 26        |
| 3.4.3    | Intuition de la méthode . . . . .                     | 27        |
| 3.4.4    | Algorithme . . . . .                                  | 27        |
| 3.4.5    | Pourquoi la méthode converge-t-elle ? . . . . .       | 27        |
| 3.4.6    | Différence d'esprit entre les deux méthodes . . . . . | 27        |
| 3.5      | Exemple . . . . .                                     | 28        |
| 3.6      | Transition vers le model-free RL . . . . .            | 29        |
|          | <b>Conclusion</b>                                     | <b>31</b> |
|          | <b>Références</b>                                     | <b>32</b> |

# Avant-propos

Ces notes introduisent l'apprentissage par renforcement, ou *reinforcement learning* (RL), avec un double objectif : d'une part présenter le cadre conceptuel général, d'autre part montrer comment ce cadre conduit naturellement à des méthodes de programmation dynamique lorsque le modèle de l'environnement est connu.

Le RL se distingue des autres formes d'apprentissage par le fait qu'il traite de *décisions séquentielles*. L'agent n'a pas seulement à produire une bonne sortie pour une entrée donnée ; il doit choisir des actions qui modifient le futur. Une décision raisonnable à court terme peut être catastrophique à long terme, et inversement. C'est cette dépendance temporelle qui rend le domaine à la fois fascinant et subtil.

On supposera connus les outils mathématiques élémentaires des classes préparatoires scientifiques : probabilités discrètes, matrices, suites, espérance, maximum. En revanche, aucune connaissance préalable en apprentissage automatique n'est nécessaire.

# Notations globales

Les notations suivantes seront utilisées de manière uniforme dans tout le document.

---

|                     |                                                                 |
|---------------------|-----------------------------------------------------------------|
| $\mathcal{S}$       | ensemble des états ( <i>states</i> );                           |
| $\mathcal{A}$       | ensemble des actions ( <i>actions</i> );                        |
| $S_t$               | état à l'instant discret $t$ ;                                  |
| $A_t$               | action choisie à l'instant $t$ ;                                |
| $R_{t+1}$           | récompense reçue après l'action $A_t$ ;                         |
| $\pi$               | politique ( <i>policy</i> );                                    |
| $\gamma \in [0, 1)$ | facteur d'actualisation ( <i>discount factor</i> );             |
| $P(s'   s, a)$      | probabilité de transition de $s$ vers $s'$ sous l'action $a$ ;  |
| $r(s, a, s')$       | récompense associée à la transition $(s, a, s')$ ;              |
| $v_\pi(s)$          | valeur d'un état $s$ sous la politique $\pi$ ;                  |
| $q_\pi(s, a)$       | valeur du couple état-action $(s, a)$ sous la politique $\pi$ . |

---

Le temps sera toujours discret :  $t = 0, 1, 2, \dots$ . La récompense est notée  $R_{t+1}$  pour rappeler qu'elle est observée *après* le choix de l'action  $A_t$ .

# Chapitre 1

## Motivations et caractéristiques de l'apprentissage par renforcement

### 1.1 Pourquoi un cadre spécifique ?

L'apprentissage par renforcement est le cadre naturel pour étudier des problèmes où un système interagit avec son milieu, agit de manière répétée et cherche à maximiser une performance globale dans le temps. Ce point est essentiel : en RL, la qualité d'une action ne peut pas toujours être jugée uniquement par son effet immédiat. Il faut souvent considérer ses conséquences futures.

Dans l'apprentissage supervisé, on dispose d'exemples du type "entrée → bonne sortie". Dans le RL, ce type d'information détaillée n'est généralement pas disponible. L'agent n'est pas guidé pas à pas ; il reçoit seulement des indications sur la qualité de ce qui se passe, sous forme de récompenses. Il doit donc apprendre à relier ses décisions présentes aux conséquences ultérieures.

Ce cadre apparaît dès que l'on doit choisir une suite d'actions : piloter un robot, jouer à un jeu, gérer un réseau, déplacer un véhicule, allouer des ressources, régler un procédé industriel ou optimiser une stratégie sur plusieurs étapes.

### 1.2 L'agent

**Définition 1.1** (Agent). Un **agent** (*agent*) est l'entité qui prend les décisions. À chaque étape, il reçoit une information sur la situation courante, choisit une action, puis modifie éventuellement son comportement à partir du retour fourni par l'environnement.

Le terme est volontairement général. Un agent peut être :

- un programme contrôlant un robot ;
- un algorithme de jeu ;
- un contrôleur automatique dans un système physique ;
- plus abstraitement, une règle mathématique transformant une observation en action.

Le point important n'est pas la nature matérielle de l'agent, mais son rôle : *choisir*.

### 1.3 L'environnement

**Définition 1.2** (Environnement). L'**environnement** (*environment*) désigne tout ce qui n'est pas l'agent : les règles du système, la dynamique du monde extérieur, les perturbations aléatoires, la manière dont les

observations sont produites et celle dont les récompenses sont attribuées.

L'agent et l'environnement forment une boucle. L'agent agit ; l'environnement répond. Cette boucle élémentaire suffit déjà à faire apparaître la structure du RL.

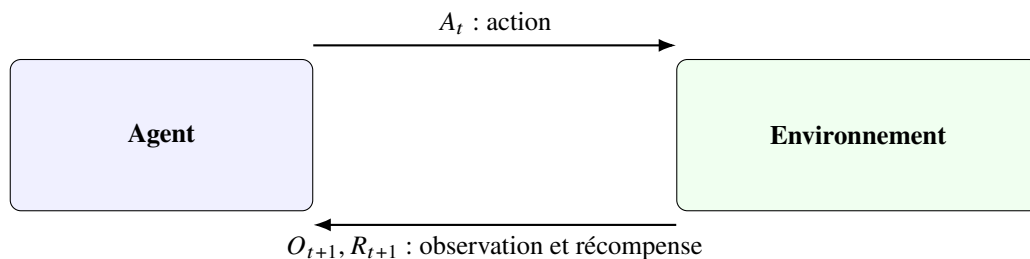


FIGURE 1.1 – Boucle d'interaction élémentaire en apprentissage par renforcement.

Dans cette première présentation, on note  $O_t$  l'observation disponible au temps  $t$ . Plus tard, lorsque l'on passera au cadre markovien, cette information sera résumée par un état  $S_t$ .

## 1.4 Le feedback

Le mot **feedback** désigne l'information renvoyée par l'environnement en réponse aux actions de l'agent. En RL, ce feedback est en général *évaluatif* plutôt qu'*instructif*.

Cela signifie qu'il ne dit pas nécessairement quelle action était correcte. Il indique seulement si la situation semble meilleure ou pire. L'agent ne reçoit pas : "il fallait choisir l'action 3", mais plutôt : "ce qui vient de se passer semble positif" ou "ce qui vient de se passer semble négatif".

Cette différence change profondément la nature du problème. L'agent doit à la fois :

1. évaluer les conséquences de ses actions ;
2. apprendre à prendre de meilleures décisions à partir de cette évaluation.

Le feedback peut être fréquent ou rare, précis ou bruité, immédiat ou retardé. Plus il est pauvre ou tardif, plus l'apprentissage est difficile.

## 1.5 Récompense immédiate et récompense retardée

**Définition 1.3** (Récompense). La **récompense** (*reward*) est un signal scalaire, généralement noté  $R_{t+1}$ , envoyé par l'environnement après une transition. Elle quantifie localement la qualité de ce qui vient de se produire.

Le fait que la récompense soit un scalaire est crucial. Un objectif éventuellement complexe doit être ramené à un nombre réel. Toute la difficulté de modélisation consiste alors à faire en sorte que ce nombre représente correctement le but poursuivi.

On distingue conceptuellement deux situations.

### Récompense immédiate

Une récompense est dite *immédiate* lorsque la qualité de l'action apparaît tout de suite. Dans un jeu d'arcade, ramasser un bonus peut rapporter immédiatement des points ; le lien entre action et gain est alors simple à identifier.

## Récompense retardée

Une récompense est dite *retardée* lorsque l'effet réel d'une action n'apparaît qu'après plusieurs étapes. C'est la situation la plus intéressante et la plus difficile. Dans un jeu stratégique, un coup peut sembler neutre aujourd'hui tout en préparant une position excellente plus tard.

Ce décalage temporel mène au problème classique de *l'assignation du crédit (credit assignment)* : lorsqu'une bonne ou une mauvaise conséquence survient tardivement, comment savoir quelles décisions passées en sont responsables ?

## 1.6 Exemples de rewards

**Exemple 1.4** (Robot mobile). Un robot doit atteindre une cible dans une pièce. On peut définir

$$R_{t+1} = \begin{cases} +100 & \text{si la cible est atteinte,} \\ -100 & \text{si un obstacle est percuté,} \\ -1 & \text{sinon à chaque pas.} \end{cases}$$

La pénalité constante  $-1$  sert à encourager les trajectoires courtes.

**Exemple 1.5** (Jeu Atari). Dans un jeu Atari, la récompense peut être tout simplement l'augmentation du score affiché à l'écran. Maximiser la somme des récompenses revient alors à maximiser le score total.

**Exemple 1.6** (Jeu de Go). Une modélisation minimale consiste à donner  $+1$  en cas de victoire,  $-1$  en cas de défaite et  $0$  à tous les autres instants. Le signal est alors extrêmement retardé : l'information utile n'arrive qu'à la fin de la partie.

**Exemple 1.7** (Conduite d'un véhicule). La récompense peut combiner plusieurs objectifs : sécurité, maintien dans la voie, confort, sobriété énergétique, progression vers la destination. On obtient alors une somme pondérée de termes distincts.

Ces exemples montrent qu'une récompense n'est pas une simple décoration mathématique. Elle encode concrètement ce que l'on veut que l'agent apprenne à faire.

## 1.7 Types d'environnement

Une distinction très importante concerne ce que l'on connaît de la dynamique de l'environnement.

### Environnement complètement connu

On parle d'environnement *complètement connu* lorsque les règles d'évolution sont données. Pour chaque état et chaque action, on connaît les transitions possibles et leurs probabilités, ainsi que les récompenses associées. On dispose alors d'un *modèle*.

Ce cas est typique des exercices théoriques et de certains problèmes industriels bien formalisés. Il permet l'utilisation directe de la programmation dynamique.

### Environnement partiellement connu ou inconnu

Dans beaucoup de situations réelles, on ne connaît pas à l'avance les probabilités de transition ni même la structure exacte des récompenses. L'agent doit alors apprendre à partir de l'expérience. C'est le cadre des méthodes *model-free* ou des approches d'apprentissage de modèle.

## 1.8 Schéma classique d'interaction en RL

Sans encore parler de processus de décision markovien, on peut décrire l'apprentissage par renforcement comme la production d'une suite de données. Au temps  $t$  :

1. l'agent reçoit une observation  $O_t$  ;
2. il choisit une action  $A_t$  ;
3. l'environnement répond ;
4. l'agent reçoit une nouvelle observation  $O_{t+1}$  et une récompense  $R_{t+1}$ .

On obtient donc une trajectoire du type

$$O_0, A_0, R_1, O_1, A_1, R_2, O_2, \dots$$

Ou, lorsque l'observation pertinente peut être résumée par un état,

$$S_0, A_0, R_1, S_1, A_1, R_2, S_2, \dots$$

Il est fondamental de remarquer que ces données ne sont pas indépendantes et ne sont pas fournies à l'avance. Elles sont *produites par les actions de l'agent*. L'agent influence donc les données sur lesquelles il apprend.

C'est ce qui mène au dilemme central entre :

- **exploration** (*exploration*) : essayer des actions encore mal connues ;
- **exploitation** (*exploitation*) : choisir ce qui semble déjà le meilleur.

## 1.9 Quels problèmes peut-on traiter ?

Le RL est particulièrement adapté lorsque plusieurs caractéristiques sont présentes simultanément :

- le problème comporte une suite de décisions ;
- une action modifie les possibilités futures ;
- il n'existe pas d'étiquette complète donnant la bonne action en chaque situation ;
- l'objectif porte sur la performance globale dans le temps et non sur le seul instant présent.

Sur le plan mathématique, on se situe dans le domaine du contrôle séquentiel. Sur le plan algorithmique, le RL forme un pont entre probabilités, optimisation, apprentissage statistique et théorie de la décision.

## 1.10 Exemples introductifs

### Robot de navigation

Un robot se déplace dans un bâtiment. À chaque étape, il choisit entre plusieurs mouvements élémentaires. Son objectif est d'atteindre une destination tout en évitant les obstacles. Le RL est pertinent car les choix faits maintenant déterminent les états futurs dans lesquels le robot se trouvera.

### Jeux Atari

Dans beaucoup de jeux Atari, l'agent reçoit une image de l'écran et doit choisir parmi quelques actions discrètes. Même si les actions sont simples, la difficulté vient du fait que le bon comportement se juge sur une longue séquence d'interactions.

## Jeu de Go

Le Go illustre la difficulté de l'exploration. L'espace des coups est immense, les parties sont longues et la récompense finale est retardée. Cela oblige l'agent à raisonner très au-delà du court terme.

## Autres exemples

On peut encore penser à :

- la gestion d'un ascenseur ;
- l'ordonnancement dynamique de tâches ;
- le réglage automatique d'une température ;
- la planification d'itinéraires ;
- certaines stratégies d'interaction homme-machine.

Tous ces exemples partagent la même idée : *agir aujourd'hui modifie ce qui sera possible demain*. C'est précisément cela que le RL cherche à modéliser.

## 1.11 Quelques repères historiques

L'apprentissage par renforcement n'est pas apparu d'un seul coup : il s'est construit progressivement, à la rencontre du contrôle optimal, de la théorie de la décision, de l'intelligence artificielle et de la psychologie de l'apprentissage.

Un premier jalon fondamental est la **programmation dynamique** de Bellman, dans les années 1950. L'idée centrale – décomposer un problème séquentiel en sous-problèmes plus simples – est au coeur de toute la théorie moderne du RL. Les équations de Bellman, vues dans ce cours, en sont l'expression mathématique la plus importante.

Très tôt, des programmes capables d'apprendre par interaction apparaissent dans des jeux. L'exemple classique est celui du programme de dames d'Arthur Samuel, développé dans les années 1950 et publié en 1959. Ce travail est souvent présenté comme l'un des premiers exemples marquants d'apprentissage automatique : le programme améliore progressivement son jeu à partir de son expérience.

Dans les années 1980, le domaine prend une forme plus théorique. Les idées d'**apprentissage par différence temporelle** (*temporal-difference learning*) et d'**actor-critic** se développent, puis la fin des années 1980 voit apparaître **Q-learning**, méthode majeure car elle permet d'apprendre une politique optimale sans connaître à l'avance le modèle de l'environnement. À partir de ce moment, le RL moderne se structure autour de notions devenues classiques : fonction de valeur, politique, compromis exploration/exploitation, et apprentissage *model-free*.

Dans les années 1990, le succès de **TD-Gammon** montre que ces idées peuvent produire des comportements de très haut niveau dans un jeu difficile comme le backgammon. Ce résultat a beaucoup contribué à convaincre la communauté que le RL n'était pas seulement élégant en théorie, mais aussi réellement puissant en pratique.

Un autre tournant majeur arrive en 2015 avec **DQN** (*Deep Q-Network*) : pour la première fois, un même algorithme parvient à apprendre directement à partir des pixels sur un grand ensemble de jeux Atari. C'est l'un des actes fondateurs du **deep reinforcement learning**, c'est-à-dire de la rencontre entre RL et réseaux de neurones profonds.

Les années suivantes voient des démonstrations spectaculaires dans les jeux de stratégie. **AlphaGo** bat un champion humain au jeu de Go en 2016, puis **AlphaZero** montre qu'un même schéma général peut apprendre par auto-jeu à très haut niveau dans plusieurs jeux classiques. **MuZero** franchit une étape

supplémentaire en combinant apprentissage et planification, sans recevoir explicitement les règles du jeu sous forme d'un modèle donné à l'avance.

Enfin, plus récemment, certaines idées du RL ont aussi joué un rôle important dans l'**alignement des modèles de langage**, en particulier avec le *reinforcement learning from human feedback* (RLHF), où un signal de préférence humaine sert à guider l'apprentissage. Cela montre que le RL dépasse largement le cadre des jeux : il constitue aujourd'hui une des grandes approches générales pour apprendre à décider dans des problèmes séquentiels complexes.

Pour ce cours, on retiendra surtout l'idée suivante : historiquement, le RL est passé d'un cadre mathématique de décision séquentielle à un ensemble de méthodes capables d'apprendre par interaction, puis à des systèmes puissants combinant estimation de valeur, optimisation de politique, approximation par réseaux de neurones et parfois planification.

## 1.12 Applications pratiques actuelles du RL

Aujourd'hui, les applications les plus concrètes du RL apparaissent surtout dans les problèmes où il faut prendre une suite de décisions en tenant compte d'un effet à long terme. On le rencontre par exemple dans la personnalisation et les systèmes de recommandation, où l'on cherche non seulement à maximiser un clic immédiat, mais aussi la satisfaction future de l'utilisateur. Le RL est également utilisé dans certains problèmes de contrôle et d'optimisation de systèmes physiques ou industriels, par exemple lorsque l'on veut réduire une consommation d'énergie ou piloter plus efficacement un processus complexe. Il joue aussi un rôle croissant en robotique, où l'objectif est d'apprendre des comportements adaptés dans des environnements physiques incertains. Enfin, une application aujourd'hui particulièrement visible est l'alignement des grands modèles de langage : des préférences humaines peuvent être utilisées comme signal de récompense pour guider le modèle vers des réponses plus utiles, plus sûres et plus conformes aux attentes.

# Chapitre 2

## Cadre théorique

### 2.1 État markovien

Pour traiter mathématiquement les décisions séquentielles, il faut disposer d'une description synthétique de la situation courante. C'est le rôle de l'état.

**Définition 2.1** (État markovien, *Markov state*). Un état  $S_t$  est dit **markovien** si, connaissant  $S_t$  et l'action  $A_t$ , la distribution du prochain état et de la prochaine récompense ne dépend plus du passé plus ancien. Formellement,

$$\mathbb{P}(S_{t+1} = s', R_{t+1} = r \mid S_t, A_t, S_{t-1}, A_{t-1}, \dots, S_0, A_0) = \mathbb{P}(S_{t+1} = s', R_{t+1} = r \mid S_t, A_t).$$

L'idée n'est pas que le système n'a pas de mémoire, mais que *toute la mémoire utile a été condensée dans l'état*. Si ce résumé est correct, alors l'avenir dépend seulement de la situation actuelle et de l'action choisie, pas du détail complet du passé.

**Remarque 2.2.** Dans certains problèmes, l'observation brute n'est pas markovienne. Par exemple, une seule image d'écran peut ne pas permettre de déduire une vitesse. Il faut alors enrichir l'état, par exemple en empilant plusieurs observations successives.

### 2.2 Markov Reward Process

Avant d'introduire les actions, il est utile d'étudier le cas où la manière d'agir est déjà fixée.

**Définition 2.3** (Markov Reward Process, MRP). Un **processus de récompense markovien** (*Markov Reward Process*, MRP) est un triplet

$$(\mathcal{S}, P, r)$$

où :

- $\mathcal{S}$  est un ensemble fini d'états ;
- $P = (P(s, s'))_{s, s' \in \mathcal{S}}$  est une matrice de transition avec

$$P(s, s') = \mathbb{P}(S_{t+1} = s' \mid S_t = s);$$

- $r : \mathcal{S} \rightarrow \mathbb{R}$  est une fonction donnant la récompense immédiate moyenne

$$r(s) = \mathbb{E}[R_{t+1} \mid S_t = s].$$

Un MRP décrit donc une évolution stochastique sans contrôle explicite. Il constitue le bon cadre pour définir la valeur lorsque la règle de décision est déjà imposée.

## 2.3 Matrice de transition

**Définition 2.4** (Matrice de transition, *transition matrix*). Une **matrice de transition** est une matrice carrée  $P = (P(s, s'))$  telle que pour tous les états  $s, s'$  :

$$P(s, s') \geq 0 \quad \text{et} \quad \sum_{s' \in \mathcal{S}} P(s, s') = 1.$$

Chaque ligne représente donc une loi de probabilité sur l'état suivant.

Lorsque des actions sont disponibles, une seule matrice ne suffit plus. Il faut une famille de probabilités conditionnelles

$$P(s' | s, a),$$

qui indique comment l'action  $a$  influence la transition depuis l'état  $s$ .

## 2.4 Politique et MDP

Pour introduire le contrôle, on ajoute les actions au cadre markovien.

**Définition 2.5** (Processus de décision markovien, *Markov Decision Process*). Un **processus de décision markovien** (*Markov Decision Process*, MDP) fini est un quadruplet

$$(\mathcal{S}, \mathcal{A}, P, r)$$

où :

- $\mathcal{S}$  est l'ensemble des états ;
- $\mathcal{A}$  est l'ensemble des actions ;
- $P(s' | s, a)$  est la probabilité de transition vers  $s'$  après l'action  $a$  en  $s$  ;
- $r(s, a, s')$  est la récompense associée à cette transition.

On y ajoute en pratique un facteur d'actualisation  $\gamma \in [0, 1)$ .

**Définition 2.6** (Politique, *policy*). Une **politique** est une règle de décision. Mathématiquement, une politique stationnaire est une application donnant, pour chaque état  $s$ , une distribution de probabilité sur les actions :

$$\pi(a | s) = \mathbb{P}(A_t = a | S_t = s).$$

Si une seule action reçoit la probabilité 1 dans chaque état, la politique est dite déterministe.

Une politique transforme un MDP en MRP. En effet, une fois les décisions fixées par  $\pi$ , la dynamique globale dépend seulement des états.

**Rappel (math/stat) :** Soit une variable aléatoire discrète  $X$  qui peut prendre les valeurs  $(x_1, x_2, \dots, x_n)$  avec les probabilités  $(p_1, p_2, \dots, p_n)$  (vecteur nommé *distribution* de  $X$ , ou *densité de probabilité* de  $X$ ). La moyenne de  $X$ , appelée aussi *espérance mathématique* de  $X$ , est donné par la relation :

$$E(X) = p_1 x_1 + p_2 x_2 + \dots + p_n x_n = \sum_{i=1}^n p_i x_i$$

Sous une politique  $\pi$ , la matrice de transition induite est donné par la moyenne/espérance des probabilités de transition pour toutes les actions :

$$P_\pi(s, s') = \sum_{a \in \mathcal{A}} \pi(a | s) P(s' | s, a),$$

et la récompense moyenne immédiate devient :

$$r_{\pi}(s) = \sum_{a \in \mathcal{A}} \pi(a | s) \sum_{s' \in \mathcal{S}} P(s' | s, a) r(s, a, s').$$

## 2.5 Retour cumulé

Optimiser seulement la récompense immédiate serait une stratégie myope. On cherche en réalité une récompense cumulative.

**Définition 2.7** (Retour, *return*, *gain*). Le **retour** à partir de l'instant  $t$  est la variable aléatoire

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+1+k},$$

où  $\gamma \in [0, 1)$  est le facteur d'actualisation.

Le facteur  $\gamma$  a plusieurs interprétations utiles :

- il valorise davantage les récompenses proches que les récompenses lointaines ;
- il stabilise mathématiquement la somme lorsque les récompenses sont bornées ;
- il modélise une préférence pour le court terme ou une incertitude croissante sur le futur lointain.

Quand  $\gamma$  est proche de 0, l'agent voit très peu au-delà du prochain instant. Quand  $\gamma$  est proche de 1, il tient fortement compte du long terme.

## 2.6 Fonction de valeur d'état $v_{\pi}$

**Définition 2.8** (Fonction de valeur d'état, *state-value function*). Soit  $\pi$  une politique. La **fonction de valeur d'état** associée à  $\pi$  est la fonction

$$v_{\pi} : \mathcal{S} \rightarrow \mathbb{R}, \quad v_{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t = s].$$

Elle représente la récompense cumulée espérée lorsqu'on part de l'état  $s$  et que l'on suit ensuite la politique  $\pi$ .

Cette définition contient plusieurs informations à ne pas mélanger.

- Le retour  $G_t$  est une somme de récompenses futures.
- Le symbole  $\mathbb{E}_{\pi}$  signifie que les actions sont choisies selon la politique  $\pi$ .
- On conditionne par l'événement "on est actuellement dans l'état  $s$ ".

La valeur n'est donc pas une récompense observée sur une trajectoire unique, mais une moyenne théorique sur toutes les trajectoires compatibles avec la politique et la dynamique du système.

## 2.7 Fonction de valeur d'état-action $q_{\pi}$

**Définition 2.9** (Fonction de valeur d'état-action, *action-value function*). Soit  $\pi$  une politique. La **fonction de valeur d'état-action** est définie par

$$q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t | S_t = s, A_t = a].$$

Elle mesure la récompense cumulée espérée si l'on se trouve dans l'état  $s$ , que l'on choisit d'abord l'action  $a$ , puis que l'on suit la politique  $\pi$  par la suite.

Conceptuellement :

- $v_\pi(s)$  évalue l'état  $s$  si l'on suit déjà  $\pi$  ;
- $q_\pi(s, a)$  évalue spécifiquement l'action  $a$  à l'intérieur de l'état  $s$ .

Cette seconde fonction est particulièrement utile pour comparer les actions et améliorer une politique.

## 2.8 Valeurs optimales

Le but final est de trouver une politique aussi bonne que possible.

**Définition 2.10** (Fonctions de valeur optimales). La **valeur optimale d'état** (*optimal state-value function*) est définie par

$$v_*(s) = \sup_{\pi} v_{\pi}(s).$$

La **valeur optimale d'état-action** (*optimal action-value function*) est définie par

$$q_*(s, a) = \sup_{\pi} q_{\pi}(s, a).$$

Dans les MDP finis avec  $\gamma < 1$ , ces bornes supérieures sont atteintes par au moins une politique stationnaire déterministe optimale. On peut donc parler d'une politique  $\pi_*$  telle que

$$v_{\pi_*}(s) = v_*(s) \quad \text{pour tout } s \in \mathcal{S}.$$

## 2.9 Valeurs optimales

Le but final est de déterminer une politique qui maximise le retour cumulé attendu. Les définitions suivantes sont *ponctuelles* : elles sont faites état par état, ou couple état-action par couple état-action.

**Définition 2.11** (Fonctions de valeur optimales). La **valeur optimale d'état** (*optimal state-value function*) est définie, pour tout état  $s \in \mathcal{S}$ , par

$$v_*(s) = \sup_{\pi} v_{\pi}(s).$$

La **valeur optimale d'état-action** (*optimal action-value function*) est définie, pour tout couple admissible  $(s, a)$ , par

$$q_*(s, a) = \sup_{\pi} q_{\pi}(s, a).$$

Ces définitions doivent être lues avec soin. La quantité  $v_*(s)$  est le meilleur retour espéré que l'on puisse obtenir en partant de l'état  $s$ . De même,  $q_*(s, a)$  est le meilleur retour espéré que l'on puisse obtenir en partant de l'état  $s$ , en imposant d'abord l'action  $a$ , puis en agissant de manière optimale par la suite.

Autrement dit, les optimisations ci-dessus sont faites *point par point*. A priori, la politique qui réalise le supremum dans la définition de  $v_*(s)$  pourrait dépendre de l'état  $s$ , et la politique qui réalise le supremum dans la définition de  $q_*(s, a)$  pourrait dépendre du couple  $(s, a)$ .

Il faut donc distinguer deux idées :

- les fonctions  $v_*$  et  $q_*$ , qui sont définies localement, état par état ou couple par couple ;
- l'existence éventuelle d'une *même* politique qui soit optimale simultanément pour tous les états.

Dans le cadre standard des **MDP finis** (*finite Markov decision processes*), avec **facteur d'actualisation** (*discount factor*) vérifiant

$$0 \leq \gamma < 1,$$

et avec récompenses bornées, on dispose des résultats fondamentaux suivants.

**Propriété 2.12** (Existence d’une politique optimale stationnaire déterministe). Il existe au moins une politique stationnaire déterministe  $\pi_*$  telle que

$$v_{\pi_*}(s) = v_*(s) \quad \text{pour tout } s \in \mathcal{S}.$$

Autrement dit, bien que  $v_*(s)$  soit défini séparément pour chaque état, il existe une *même* politique qui atteint simultanément l’optimum dans tous les états.

**Propriété 2.13** (Lien entre  $v_*$  et  $q_*$ ). Pour tout état  $s \in \mathcal{S}$ ,

$$v_*(s) = \max_{a \in \mathcal{A}} q_*(s, a).$$

Ainsi, la meilleure valeur atteignable depuis un état est obtenue en choisissant, dans cet état, une action qui maximise la valeur optimale d’état-action.

**Propriété 2.14** (Caractérisation des politiques optimales). Une politique stationnaire déterministe  $\pi_*$  est optimale si, et seulement si,

$$\pi_*(s) \in \arg \max_{a \in \mathcal{A}} q_*(s, a) \quad \text{pour tout } s \in \mathcal{S}.$$

Dans ce cas, on a non seulement

$$v_{\pi_*}(s) = v_*(s) \quad \text{pour tout } s \in \mathcal{S},$$

mais aussi

$$q_{\pi_*}(s, a) = q_*(s, a) \quad \text{pour tout couple } (s, a).$$

Ces résultats justifient le vocabulaire de **politique optimale** (*optimal policy*) : dans les MDP finis actualisés, il ne s’agit pas d’une politique optimale seulement pour un état particulier, mais d’une politique qui est optimale de manière cohérente sur tout l’espace d’états.

Il est important de ne pas mal interpréter cette propriété. Les fonctions  $v_*(s)$  et  $q_*(s, a)$  ne sont évidemment pas indépendantes de  $s$  : elles dépendent au contraire de l’état de départ, et pour  $q_*$  aussi de l’action initiale. Ce qui devient indépendant du choix de l’état, sous les hypothèses précédentes, c’est le fait qu’une *même politique* peut réaliser tous ces optima simultanément.

Enfin, l’optimalité n’implique pas en général l’unicité. Il peut exister plusieurs politiques optimales distinctes, par exemple lorsque plusieurs actions donnent exactement la même valeur optimale dans un état donné. On parle donc plus rigoureusement d’une politique optimale que de la politique optimale.

## 2.10 Équations de Bellman

Les équations de Bellman traduisent une idée extrêmement simple et fondamentale : la valeur présente de  $v_\pi(s)$  ou  $q_\pi(s, a)$  dans l’état courant  $s_t = s$  est égale à l’espérance de la somme entre la récompense immédiate et la valeur actualisée du futur dans l’état suivant  $s_{t+1} = s'$ . Plus précisément : la valeur d’un état, ou d’un couple état-action, se décompose en deux termes : la récompense immédiate attendue, puis la valeur actualisée attendue de la suite du processus.

Autrement dit, si l’on se trouve à l’instant  $t$  dans l’état  $S_t = s$ , alors la valeur sous la politique  $\pi$  est obtenue en additionnant la récompense reçue au prochain pas de temps et la valeur du futur à partir de l’état suivant  $S_{t+1}$ , le tout en prenant l’espérance appropriée.

Pour la fonction de valeur d’état, le futur est mesuré par  $v_\pi(S_{t+1})$ . Pour la fonction de valeur d’état-action, si l’on impose d’abord l’action  $A_t = a$ , le futur peut être décrit soit par  $v_\pi(S_{t+1})$ , soit, de manière équivalente, par  $q_\pi(S_{t+1}, A_{t+1})$  après le choix de l’action suivante selon la politique  $\pi$ .

## Bellman pour $v_\pi$

Pour une politique fixée  $\pi$ ,

$$v_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a | s) \sum_{s' \in \mathcal{S}} P(s' | s, a) \left[ r(s, a, s') + \gamma v_\pi(s') \right].$$

Cette formule se lit en deux temps.

La somme extérieure

$$\sum_{a \in \mathcal{A}} \pi(a | s) (\dots)$$

fait la moyenne sur les actions possibles dans l'état courant  $s$ , selon la politique  $\pi$ .

Pour une action  $a$  fixée, la somme intérieure

$$\sum_{s' \in \mathcal{S}} P(s' | s, a) \left[ r(s, a, s') + \gamma v_\pi(s') \right]$$

est l'espérance de la quantité

$$r(s, a, s') + \gamma v_\pi(s')$$

par rapport à l'état suivant  $s'$ .

Autrement dit, si l'agent est actuellement dans l'état  $s$  et choisit l'action  $a$ , alors l'environnement peut conduire vers plusieurs états suivants possibles  $s'$ . Chacun de ces états suivants apparaît avec une probabilité

$$P(s' | s, a).$$

Pour chaque état suivant  $s'$ , on additionne deux contributions :

- la récompense immédiate  $r(s, a, s')$  reçue lors de la transition de  $s$  vers  $s'$  après l'action  $a$  ;
- la valeur future actualisée  $\gamma v_\pi(s')$ , qui mesure l'intérêt de continuer ensuite à suivre la politique  $\pi$  à partir du nouvel état  $s'$ .

La somme en rouge est donc une moyenne pondérée sur tous les états suivants possibles. Elle répond à la question suivante :

*Si l'on est dans l'état  $s$  et que l'on choisit l'action  $a$ , quelle est la récompense immédiate moyenne plus la valeur future moyenne actualisée, en tenant compte de tous les états suivants possibles ?*

On peut donc voir cette somme comme la **valeur attendue d'un pas d'évolution** du système après avoir choisi l'action  $a$ .

Dans le cas particulier où la transition est déterministe, c'est-à-dire lorsqu'un seul état suivant  $s'$  est possible avec probabilité 1, la somme intérieure se réduit à un seul terme :

$$r(s, a, s') + \gamma v_\pi(s').$$

Dans le cas général, elle fait la moyenne sur tous les futurs possibles. La somme en rouge représente donc l'effet de l'environnement : une fois l'action  $a$  choisie, c'est l'environnement qui détermine aléatoirement l'état suivant  $s'$ .

## Bellman pour $q_\pi$

On a aussi

$$q_\pi(s, a) = \sum_{s' \in \mathcal{S}} P(s' | s, a) \left[ r(s, a, s') + \gamma \sum_{a' \in \mathcal{A}} \pi(a' | s') q_\pi(s', a') \right].$$

## Bellman pour $q_\pi$

On a aussi

$$q_\pi(s, a) = \sum_{s' \in \mathcal{S}} P(s' | s, a) \left[ r(s, a, s') + \gamma \sum_{a' \in \mathcal{A}} \pi(a' | s') q_\pi(s', a') \right].$$

Cette équation se lit de la manière suivante.

On suppose que l'agent se trouve dans l'état  $s$  et que l'on impose l'action  $a$ . La quantité  $q_\pi(s, a)$  représente alors le retour cumulé espéré obtenu en choisissant d'abord cette action, puis en suivant ensuite la politique  $\pi$ .

La somme

$$\sum_{s' \in \mathcal{S}} P(s' | s, a) (\dots)$$

fait la moyenne sur tous les états suivants possibles  $s'$  après avoir exécuté l'action  $a$  dans l'état  $s$ .

Pour un état suivant  $s'$  fixé, le terme

$$r(s, a, s') + \gamma \sum_{a' \in \mathcal{A}} \pi(a' | s') q_\pi(s', a')$$

contient deux parties.

La première partie,

$$r(s, a, s'),$$

est la récompense immédiate reçue lors de la transition de  $s$  vers  $s'$  après l'action  $a$ .

La seconde partie,

$$\gamma \sum_{a' \in \mathcal{A}} \pi(a' | s') q_\pi(s', a'),$$

est la valeur future actualisée à partir du nouvel état  $s'$ .

Regardons-la de plus près. Une fois arrivé en  $s'$ , l'agent ne choisit pas forcément toujours la même action : il agit selon la politique  $\pi$ . Ainsi, pour chaque action possible  $a'$  dans l'état  $s'$ , la politique attribue une probabilité

$$\pi(a' | s').$$

La somme

$$\sum_{a' \in \mathcal{A}} \pi(a' | s') q_\pi(s', a')$$

est donc la moyenne des valeurs d'état-action possibles dans le nouvel état  $s'$ , pondérée par les probabilités de choix données par la politique. Autrement dit, c'est exactement la valeur d'état

$$v_\pi(s').$$

On peut donc réécrire l'expression en rouge sous la forme

$$r(s, a, s') + \gamma v_\pi(s').$$

L'équation de Bellman pour  $q_\pi$  dit donc ceci :

*Si l'on choisit d'abord l'action  $a$  dans l'état  $s$ , alors la valeur de ce choix est égale à la récompense immédiate attendue, plus la valeur future actualisée attendue du processus, en supposant qu'ensuite l'agent suive la politique  $\pi$ .*

L'intuition est la suivante. La fonction  $q_\pi(s, a)$  mesure la qualité du *premier choix*  $a$  dans l'état  $s$ . Après ce premier choix, on laisse la politique  $\pi$  reprendre la main. Le terme en rouge exprime précisément

cette idée : on reçoit d'abord une récompense immédiate, puis on ajoute ce que vaut la suite de l'histoire à partir du nouvel état atteint.

Il est utile de remarquer que cette équation sépare nettement les deux sources d'aléa du problème :

- l'environnement, à travers la transition aléatoire vers l'état suivant  $s'$  selon  $P(s' | s, a)$  ;
- la politique, à travers le choix aléatoire de l'action suivante  $a'$  selon  $\pi(a' | s')$ .

Dans le cas où la politique  $\pi$  est déterministe, la somme sur les actions suivantes se simplifie : un seul choix  $a' = \pi(s')$  a une probabilité égale à 1, et l'on obtient

$$q_{\pi}(s, a) = \sum_{s' \in \mathcal{S}} P(s' | s, a) \left[ r(s, a, s') + \gamma q_{\pi}(s', \pi(s')) \right].$$

Dans le cas général, la somme sur les actions suivantes représente une moyenne sur les décisions futures possibles de l'agent.

### Relations entre $v_{\pi}$ et $q_{\pi}$

Les deux fonctions vérifient

$$v_{\pi}(s) = \sum_{a \in \mathcal{A}} \pi(a | s) q_{\pi}(s, a)$$

et

$$q_{\pi}(s, a) = \sum_{s' \in \mathcal{S}} P(s' | s, a) \left[ r(s, a, s') + \gamma v_{\pi}(s') \right].$$

La première relation dit que la valeur d'un état est la moyenne, selon la politique, des valeurs des actions disponibles. La seconde dit qu'évaluer une action consiste à regarder ce qu'elle produit immédiatement, puis l'intérêt de l'état vers lequel elle mène.

### Bellman optimal

Pour les valeurs optimales, l'espérance sur la politique est remplacée par une maximisation :

$$v_{*}(s) = \max_{a \in \mathcal{A}} \sum_{s' \in \mathcal{S}} P(s' | s, a) \left[ r(s, a, s') + \gamma v_{*}(s') \right],$$

et

$$q_{*}(s, a) = \sum_{s' \in \mathcal{S}} P(s' | s, a) \left[ r(s, a, s') + \gamma \max_{a' \in \mathcal{A}} q_{*}(s', a') \right].$$

Ces deux équations sont au coeur de toute la théorie du RL.

## 2.11 Exemple : Grid World

Considérons un petit monde quadrillé, appelé *Grid World*, formé de quatre cases disposées en carré. Les états sont

$$\mathcal{S} = \{s_1, s_2, s_3, s_4\},$$

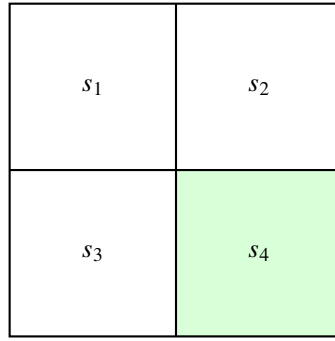
où  $s_4$  est un état terminal favorable.

Les actions disponibles dans les états non terminaux sont

$$\mathcal{A} = \{\text{haut, bas, gauche, droite}\}.$$

Lorsqu'une action voudrait faire sortir de la grille, l'agent reste sur place.

On fixe la récompense suivante :



$s_4$  : état terminal favorable

FIGURE 2.1 – Un Grid World minimal à quatre états.

- tout déplacement dans un état non terminal donne  $-1$  ;
- entrer dans  $s_4$  donne  $0$  ;
- une fois dans  $s_4$ , le processus s'arrête ou reste dans  $s_4$  avec récompense nulle.

Supposons la dynamique déterministe, pour chaque état : droite = mouvement vers la droite, gauche = mouvement vers la gauche, haut : mouvement en haut, bas : mouvement en bas. Bien sûr, si on est déjà sur le bord par rapport à la direction de mouvement, on reste en place. Une fois dans l'état  $s_4$ , on reste en  $s_4$  quelque soit l'action.

Par exemple, depuis  $s_1$ ,

$$\begin{array}{ll} \text{droite} \longrightarrow s_2, & \text{bas} \longrightarrow s_3, \\ \text{haut} \longrightarrow s_1, & \text{gauche} \longrightarrow s_1. \end{array}$$

Chaque action peut alors être représentée par une matrice de transition. Si l'on ordonne les états comme  $(s_1, s_2, s_3, s_4)$ , la matrice associée à l'action droite est

$$P^{\text{droite}} = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

Supposons maintenant que l'on choisisse la **politique uniforme** (*uniform policy*) : dans chaque état

$$\pi(a | s) = \frac{1}{4} \quad \text{pour toute action } a.$$

Sous cette politique, le MDP devient un MRP de matrice de transition

$$P_\pi = \frac{1}{4} (P^{\text{haut}} + P^{\text{bas}} + P^{\text{gauche}} + P^{\text{droite}}).$$

Avec  $\gamma = 0,9$  et  $v_\pi(s_4) = 0$ , les équations de Bellman deviennent par exemple

$$\begin{aligned} v_\pi(s_1) &= \frac{1}{4} \left[ (-1 + 0,9v_\pi(s_1)) + (-1 + 0,9v_\pi(s_1)) + (-1 + 0,9v_\pi(s_2)) + (-1 + 0,9v_\pi(s_3)) \right], \\ v_\pi(s_2) &= \frac{1}{4} \left[ (-1 + 0,9v_\pi(s_2)) + (0 + 0,9v_\pi(s_4)) + (-1 + 0,9v_\pi(s_1)) + (-1 + 0,9v_\pi(s_2)) \right], \\ v_\pi(s_3) &= \frac{1}{4} \left[ (-1 + 0,9v_\pi(s_1)) + (-1 + 0,9v_\pi(s_3)) + (-1 + 0,9v_\pi(s_3)) + (0 + 0,9v_\pi(s_4)) \right], \\ v_\pi(s_4) &= \frac{1}{4} \left[ (0 + 0,9v_\pi(s_4)) + (0 + 0,9v_\pi(s_4)) + (0 + 0,9v_\pi(s_4)) + (0 + 0,9v_\pi(s_4)) \right]. \end{aligned}$$

Comme  $s_4$  est absorbant et que toute récompense ultérieure y est nulle, on a

$$v_{\pi}(s_4) = 0 + 0,9 v_{\pi}(s_4),$$

d'où

$$v_{\pi}(s_4) = 0.$$

Cet exemple permet déjà d'identifier tous les éléments du cadre : états, actions, matrice de transition, politique, récompenses et valeurs.

## Chapitre 3

# Model-based RL : programmation dynamique

### 3.1 Problème général

Dans tout ce chapitre, on suppose que le modèle est connu, c'est-à-dire que l'on connaît :

- l'ensemble fini des états  $\mathcal{S}$  ;
- l'ensemble fini des actions  $\mathcal{A}$  ;
- les probabilités  $P(s' | s, a)$  ;
- les récompenses  $r(s, a, s')$  ;
- le facteur d'actualisation  $\gamma \in [0, 1)$ .

Le problème est alors clair : *calculer une politique optimale*. On n'a pas besoin ici d'apprendre le modèle à partir des données ; le problème est entièrement déterminé. Il reste à résoudre correctement les équations de Bellman ou à construire une procédure itérative menant à leur solution.

Les deux méthodes classiques sont *policy iteration* et *value iteration*. Elles reposent toutes deux sur le cadre théorique du chapitre précédent.

### 3.2 Évaluation d'une politique

Avant d'optimiser, il faut être capable d'évaluer une politique donnée. Soit  $\pi$  une politique fixée. Sa fonction de valeur  $v_\pi$  vérifie, pour tout état  $s$ ,

$$v_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a | s) \sum_{s' \in \mathcal{S}} P(s' | s, a) [r(s, a, s') + \gamma v_\pi(s')].$$

Supposons l'espace d'états fini,

$$\mathcal{S} = \{s_1, \dots, s_n\}.$$

On ordonne les états dans cet ordre et l'on définit :

$$v_\pi = \begin{pmatrix} v_\pi(s_1) \\ v_\pi(s_2) \\ \vdots \\ v_\pi(s_n) \end{pmatrix}, \quad r_\pi = \begin{pmatrix} r_\pi(s_1) \\ r_\pi(s_2) \\ \vdots \\ r_\pi(s_n) \end{pmatrix},$$

où

$$r_\pi(s_i) = \sum_{a \in \mathcal{A}} \pi(a | s_i) \sum_{j=1}^n P(s_j | s_i, a) r(s_i, a, s_j).$$

On définit également la matrice de transition du processus induit par la politique  $\pi$  :

$$P_\pi = \begin{pmatrix} (P_\pi)_{11} & (P_\pi)_{12} & \cdots & (P_\pi)_{1n} \\ (P_\pi)_{21} & (P_\pi)_{22} & \cdots & (P_\pi)_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ (P_\pi)_{n1} & (P_\pi)_{n2} & \cdots & (P_\pi)_{nn} \end{pmatrix},$$

avec

$$(P_\pi)_{ij} = \sum_{a \in \mathcal{A}} \pi(a | s_i) P(s_j | s_i, a).$$

Ainsi, l'équation de Bellman pour une politique fixée s'écrit sous forme matricielle :

$$v_\pi = r_\pi + \gamma P_\pi v_\pi.$$

En regroupant les termes en  $v_\pi$ , on obtient

$$(I - \gamma P_\pi) v_\pi = r_\pi.$$

Comme  $\gamma < 1$  et que  $P_\pi$  est une matrice stochastique, la matrice  $I - \gamma P_\pi$  est inversible. On peut donc écrire formellement

$$v_\pi = (I - \gamma P_\pi)^{-1} r_\pi.$$

Cette formule est importante : elle montre que, pour une politique fixée  $\pi$ , la fonction de valeur  $v_\pi$  est déterminée de manière unique.

Il faut toutefois être très précis sur l'interprétation. La résolution de ce système linéaire donne la valeur de la politique *choisie au départ*. En général, ce n'est donc *pas* encore la valeur optimale  $v_*$ . On n'obtient  $v_* = v_\pi$  que dans le cas particulier où la politique considérée est déjà optimale.

Autrement dit :

$$v_\pi = (I - \gamma P_\pi)^{-1} r_\pi \quad \text{ne donne pas directement } v_*, \text{ sauf si } \pi = \pi_*.$$

## Comment utiliser $v_\pi$ pour améliorer la politique ?

Une fois  $v_\pi$  calculée, on peut se demander, dans chaque état  $s$ , quelle action serait la meilleure si l'on connaissait cette valeur. Pour cela, on utilise la quantité

$$q_\pi(s, a) = \sum_{s' \in \mathcal{S}} P(s' | s, a) [r(s, a, s') + \gamma v_\pi(s')].$$

Cette expression mesure la qualité de l'action  $a$  dans l'état  $s$ , si l'on suppose qu'après ce premier choix on suit ensuite la politique  $\pi$ .

On peut alors définir une nouvelle politique  $\pi'$  en choisissant, dans chaque état, une action qui maximise cette quantité :

$$\pi'(s) \in \arg \max_{a \in \mathcal{A}(s)} q_\pi(s, a).$$

Cette nouvelle politique est appelée une **politique gloutonne** (*greedy policy*) par rapport à  $v_\pi$ . L'idée est simple : une fois que l'on connaît la valeur de la politique actuelle, on choisit désormais dans chaque état l'action qui paraît la plus avantageuse.

Si la politique  $\pi$  était déjà optimale, alors cette amélioration ne change rien : on retrouve une politique optimale et

$$v_\pi = v_*.$$

Dans le cas contraire, cette nouvelle politique est en général meilleure que la précédente.

## Limites de l'approche directe par résolution matricielle

La formule

$$v_\pi = (I - \gamma P_\pi)^{-1} r_\pi$$

est élégante et théoriquement très utile, mais elle présente plusieurs inconvénients pratiques.

D'abord, elle suppose que l'on construise explicitement la matrice  $P_\pi$ , de taille  $n \times n$ , où  $n = |\mathcal{S}|$  est le nombre d'états. Lorsque  $n$  est grand, cela devient coûteux en mémoire. Même lorsque la matrice est creuse, son stockage et sa manipulation peuvent être lourds.

Ensuite, résoudre exactement un système linéaire de taille  $n$  est une opération coûteuse. Les méthodes classiques d'algèbre linéaire ont un coût élevé lorsque  $n$  augmente. Dans de grands problèmes, ce coût devient prohibitif.

De plus, l'inversion explicite d'une matrice n'est généralement pas une bonne pratique numérique. En calcul scientifique, on évite autant que possible de calculer  $(I - \gamma P_\pi)^{-1}$  de manière explicite, car cela peut être coûteux et numériquement moins stable que la résolution directe du système linéaire

$$(I - \gamma P_\pi)v_\pi = r_\pi.$$

Enfin, dans de nombreux problèmes d'apprentissage par renforcement, l'espace d'états est immense, voire pratiquement infini. Par exemple :

- dans un jeu complexe, le nombre de configurations possibles peut être astronomique ;
- dans un problème robotique, l'état peut contenir des positions, vitesses, angles et mesures continues ;
- dans des environnements visuels, un état peut être une image ou une séquence d'images.

Dans de telles situations, former et résoudre un système matriciel exact n'est plus réaliste. L'approche directe est donc surtout utile :

- pour de petits problèmes ;
- pour l'analyse théorique ;
- pour comprendre la structure des équations de Bellman.

## Pourquoi introduire des méthodes itératives ?

Pour contourner ces difficultés, on préfère souvent des méthodes itératives. L'idée générale est de construire progressivement une bonne approximation des valeurs, sans jamais inverser de matrice.

Ces méthodes présentent plusieurs avantages :

- elles exploitent directement la structure récursive des équations de Bellman ;
- elles évitent l'inversion explicite de matrices de grande taille ;
- elles sont souvent plus simples à mettre en oeuvre ;
- elles peuvent être arrêtées avant convergence complète, ce qui permet déjà d'obtenir une approximation utile ;
- elles se généralisent plus naturellement aux méthodes approchées et au *model-free RL*.

Les deux méthodes fondamentales de programmation dynamique sont les suivantes.

**Policy Iteration.** On part d'une politique initiale  $\pi_0$ . On répète itérativement la procédure suivante : (1) évaluation de la politique (calcul de  $v_{\pi_k}$ ) et ensuite (2) amélioration gloutonne de la politique. Sous des hypothèses standards (MDP fini,  $\gamma < 1$ ), cette procédure converge vers une politique optimale.

**Value Iteration.** Au lieu d'évaluer exactement une politique à chaque étape, on met directement à jour une estimation de la valeur optimale par l'opérateur de Bellman optimal :

$$v_{k+1}(s) = \max_{a \in \mathcal{A}(s)} \sum_{s' \in \mathcal{S}} P(s' | s, a) [r(s, a, s') + \gamma v_k(s')].$$

Cette méthode combine en quelque sorte l'évaluation et l'amélioration en une seule mise à jour. Elle converge vers  $v_*$  sous les mêmes hypothèses.

Ainsi, la résolution directe du système linéaire permet de bien comprendre l'évaluation d'une politique, mais ce sont les méthodes itératives qui deviennent les outils naturels dès que les problèmes prennent de l'ampleur. Ces méthodes sont décrites par la suite.

### 3.3 Policy Iteration

#### 3.3.1 Quel problème essaie-t-on de résoudre ?

On cherche une politique optimale à partir d'une politique initiale quelconque  $\pi_0$ . La méthode procède en alternant deux étapes :

1. **évaluation de politique** : calculer  $v_{\pi_k}$  ;
2. **amélioration de politique** : fabriquer une nouvelle politique  $\pi_{k+1}$  qui soit gloutonne par rapport à  $v_{\pi_k}$ .

Les données de départ sont donc le MDP et une politique initiale. La sortie recherchée est une politique optimale  $\pi_*$  et, par conséquent, la valeur optimale  $v_*$ .

#### 3.3.2 Étape mathématique d'évaluation

Pour la politique courante  $\pi_k$ , les équations de Bellman forment un système d'équations linéaires. On met à jour itérativement les valeurs de chaque état jusqu'à stabilisation :

$$v_{\pi_k}(s) = \sum_{a \in \mathcal{A}} \pi_k(a | s) \sum_{s' \in \mathcal{S}} P(s' | s, a) [r(s, a, s') + \gamma v_{\pi_k}(s')].$$

Si la politique est déterministe, donc si elle choisit une action  $a_k(s)$  dans chaque état, cela devient

$$v_{\pi_k}(s) = \sum_{s' \in \mathcal{S}} P(s' | s, a_k(s)) [r(s, a_k(s), s') + \gamma v_{\pi_k}(s')].$$

**Note pour l'étape mathématique d'évaluation.** Pour la politique courante  $\pi_k$ , on cherche donc à calculer sa fonction de valeur  $v_{\pi_k}$ . Comme la politique est fixée, on n'a plus de maximisation à effectuer : les équations de Bellman deviennent un système linéaire.

Pour tout état  $s \in \mathcal{S}$ ,

$$v_{\pi_k}(s) = \sum_{a \in \mathcal{A}} \pi_k(a | s) \sum_{s' \in \mathcal{S}} P(s' | s, a) [r(s, a, s') + \gamma v_{\pi_k}(s')].$$

Il est utile d'introduire l'opérateur de Bellman associé à la politique  $\pi_k$ , noté  $T_{\pi_k}$ , défini par

$$(T_{\pi_k} v)(s) = \sum_{a \in \mathcal{A}} \pi_k(a | s) \sum_{s' \in \mathcal{S}} P(s' | s, a) [r(s, a, s') + \gamma v(s')].$$

L'évaluation de la politique consiste alors à trouver une fonction  $v$  telle que

$$v = T_{\pi_k} v.$$

Autrement dit,  $v_{\pi_k}$  est le point fixe de l'opérateur  $T_{\pi_k}$ .

Dans un MDP fini avec  $0 \leq \gamma < 1$ , on peut montrer que l'opérateur  $T_{\pi_k}$  est une contraction pour la norme infinie :

$$\|T_{\pi_k} u - T_{\pi_k} v\|_{\infty} \leq \gamma \|u - v\|_{\infty}.$$

Le théorème du point fixe de Banach garantit alors que :

- l'équation  $v = T_{\pi_k} v$  admet une unique solution ;
- en partant de n'importe quelle fonction initiale  $v^{(0)}$ , la suite définie par

$$v^{(m+1)} = T_{\pi_k} v^{(m)}$$

converge vers  $v_{\pi_k}$ .

Ainsi, l'évaluation d'une politique peut se faire de deux manières :

- soit par résolution exacte du système linéaire associé ;
- soit par itérations successives de l'opérateur  $T_{\pi_k}$  jusqu'à stabilisation.

### 3.3.3 Étape mathématique d'amélioration

Une fois  $v_{\pi_k}$  calculée, on choisit dans chaque état une action maximisant la récompense immédiate plus la valeur future estimée :

$$\pi_{k+1}(s) \in \operatorname{argmax}_{a \in \mathcal{A}} \sum_{s' \in \mathcal{S}} P(s' | s, a) [r(s, a, s') + \gamma v_{\pi_k}(s')].$$

Cette nouvelle politique est dite *gloutonne* (*greedy*) par rapport à  $v_{\pi_k}$ . Une politique gloutonne est en général prise déterministe. En cas d'égalité entre plusieurs actions maximisantes, toute sélection de l'une de ces actions définit une politique gloutonne déterministe. Cette observation implique que le processus itérative converge vers une politique déterministe.

On peut montrer que dans un MDP fini avec récompenses bornées et facteur d'actualisation  $0 \leq \gamma < 1$ , il existe au moins une politique optimale stationnaire déterministe. Cette politique peut être choisie gloutonne par rapport à  $q_*$ , c'est-à-dire telle que

$$\pi_*(s) \in \operatorname{argmax}_{a \in \mathcal{A}(s)} q_*(s, a) \quad \text{pour tout } s \in \mathcal{S}.$$

L'optimalité n'implique cependant pas l'unicité : plusieurs politiques optimales peuvent exister, et certaines peuvent être stochastiques lorsqu'il y a plusieurs actions maximisantes.

### 3.3.4 Intuition de la méthode

Pourquoi cette procédure a-t-elle une chance de marcher ? Parce qu'à chaque étape on compare, dans chaque état, l'action actuellement prescrite par la politique à toutes les autres actions possibles. Si une autre action promet une meilleure récompense immédiate plus une meilleure suite, il est rationnel de la remplacer.

Autrement dit, l'évaluation mesure la qualité de la politique courante, et l'amélioration utilise cette information pour corriger localement les choix. En répétant ce mécanisme, on élimine progressivement les mauvaises décisions.

### 3.3.5 Algorithme

#### Algorithme : Policy Iteration

**Entrée :** un MDP fini  $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$  et une politique initiale  $\pi_0$ .

1. Poser  $k = 0$ .
2. **Évaluation :** calculer exactement (ou itérativement)  $v_{\pi_k}$ .
3. **Amélioration :** pour chaque état  $s$ , choisir

$$\pi_{k+1}(s) \in \operatorname{argmax}_{a \in \mathcal{A}} \sum_{s'} P(s' | s, a) [r(s, a, s') + \gamma v_{\pi_k}(s')].$$

4. Si  $\pi_{k+1} = \pi_k$ , arrêter ; sinon remplacer  $k$  par  $k + 1$  et recommencer.

**Sortie :** une politique optimale  $\pi_*$  et sa valeur  $v_*$ .

### 3.3.6 Pourquoi est-on assuré de converger ?

Sans démonstration, les résultats essentiels sont les suivants.

**Propriété 3.1** (Résultats clés pour *policy iteration*). Dans un MDP fini avec  $\gamma < 1$  :

1. l'évaluation d'une politique admet une solution unique ;
2. l'étape d'amélioration produit une politique au moins aussi bonne que la précédente dans tous les états ;
3. s'il y a amélioration stricte dans au moins un état, on a effectivement progressé ;
4. comme le nombre de politiques déterministes stationnaires est fini, l'algorithme finit nécessairement par s'arrêter ;
5. lorsqu'il s'arrête, la politique obtenue est optimale.

Le point décisif est donc le suivant : on ne peut pas améliorer indéfiniment une politique dans un ensemble fini sans finir par atteindre une politique qui ne peut plus être améliorée. Cette politique est alors optimale.

## 3.4 Value Iteration

### 3.4.1 Quel problème essaie-t-on de résoudre ?

Dans **value iteration**, on ne cherche pas d'abord à évaluer complètement une politique. On cherche directement la valeur optimale  $v_*$ . On part d'une approximation initiale quelconque  $v_0$  et on applique répétitivement l'opérateur de Bellman optimal.

Les données de départ sont donc le MDP connu et un vecteur initial  $v_0$ . Ce que l'on veut obtenir, c'est une suite  $(v_k)$  convergeant vers  $v_*$ , puis une politique gloutonne optimale extraite de cette valeur limite.

### 3.4.2 Mise à jour mathématique

La récurrence fondamentale est

$$v_{k+1}(s) = \max_{a \in \mathcal{A}} \sum_{s' \in \mathcal{S}} P(s' | s, a) [r(s, a, s') + \gamma v_k(s')].$$

On remplace donc la valeur courante par ce que dicte l'équation de Bellman optimale en utilisant l'approximation précédente du futur.

### 3.4.3 Intuition de la méthode

On peut voir cette méthode comme une propagation progressive de l'information utile à travers l'espace des états. Si un état proche du but a une grande valeur, cette information commence à influencer les états qui peuvent le rejoindre, puis les états encore plus éloignés, et ainsi de suite.

L'algorithme n'attend pas d'avoir complètement stabilisé une politique avant de continuer. Il met à jour directement ce qui semble être la meilleure valeur possible à chaque état.

### 3.4.4 Algorithme

**Algorithme : Value Iteration**

**Entrée :** un MDP fini  $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$  et un vecteur initial  $v_0$ .

1. Pour  $k = 0, 1, 2, \dots$ , calculer pour chaque état  $s$

$$v_{k+1}(s) = \max_{a \in \mathcal{A}} \sum_{s'} P(s' | s, a) [r(s, a, s') + \gamma v_k(s')].$$

2. Arrêter lorsque la variation entre deux itérations successives est suffisamment petite.
3. Extraire une politique gloutonne par

$$\pi(s) \in \operatorname{argmax}_{a \in \mathcal{A}} \sum_{s'} P(s' | s, a) [r(s, a, s') + \gamma v_k(s')].$$

**Sortie :** une approximation de  $v_*$  et une politique gloutonne associée.

### 3.4.5 Pourquoi la méthode converge-t-elle ?

Le résultat théorique fondamental est que l'opérateur de Bellman optimal

$$(T_* v)(s) = \max_{a \in \mathcal{A}} \sum_{s' \in \mathcal{S}} P(s' | s, a) [r(s, a, s') + \gamma v(s')]$$

est une *contraction* pour la norme sup lorsque  $\gamma < 1$ . Cela signifie, en substance, que si l'on applique cet opérateur à deux fonctions de valeur différentes, elles se rapprochent d'au moins un facteur  $\gamma$ .

Autrement dit, l'itération ne s'éloigne pas de la solution : elle force progressivement les approximations à se resserrer autour d'un unique point fixe. Ce point fixe est précisément  $v_*$ .

**Propriété 3.2** (Résultats clés pour *value iteration*). Dans un MDP fini avec  $\gamma < 1$  :

1. l'opérateur de Bellman optimal possède un unique point fixe ;
2. ce point fixe est la valeur optimale  $v_*$  ;
3. pour tout choix initial  $v_0$ , la suite définie par  $v_{k+1} = T_* v_k$  converge vers  $v_*$  ;
4. toute politique gloutonne par rapport à  $v_*$  est optimale.

### 3.4.6 Différence d'esprit entre les deux méthodes

*Policy iteration* et *value iteration* résolvent le même problème, mais selon deux philosophies légèrement différentes.

- *Policy iteration* effectue de grandes étapes : on évalue complètement une politique, puis on l'améliore.

- *Value iteration* effectue des mises à jour plus élémentaires en visant directement la valeur optimale.

En pratique, la première utilise souvent moins d'itérations mais des itérations plus coûteuses, tandis que la seconde fait des itérations plus simples mais parfois plus nombreuses.

### Quand préférer l'une ou l'autre ?

Le choix entre *policy iteration* et *value iteration* dépend surtout de la taille du problème et du coût que l'on est prêt à consacrer à chaque itération.

On préfère souvent *policy iteration* lorsque l'espace d'états reste modéré et que l'on peut évaluer une politique avec une bonne précision sans coût excessif. Dans ce cas, chaque itération est plus coûteuse, car elle demande une évaluation complète de la politique courante, mais le nombre total d'itérations est souvent faible. Cette méthode est donc bien adaptée aux problèmes de taille petite ou moyenne, ou lorsque l'on cherche une politique optimale avec peu d'étapes d'amélioration.

À l'inverse, *value iteration* est souvent préférable lorsque l'évaluation exacte d'une politique devient trop coûteuse. Chaque mise à jour est plus simple, car elle applique directement l'opérateur de Bellman optimal sans passer par une résolution complète du problème d'évaluation. Cette approche est donc plus naturelle lorsque l'espace d'états est plus grand, lorsque l'on veut une méthode simple à programmer, ou lorsque l'on accepte une bonne approximation avant la convergence complète.

On peut résumer l'intuition ainsi :

- si l'on peut se permettre des étapes lourdes mais peu nombreuses, *policy iteration* est souvent un bon choix ;
- si l'on préfère des étapes légères, répétées un grand nombre de fois, *value iteration* est souvent plus adaptée.

En pratique, il existe aussi des méthodes intermédiaires, comme la *modified policy iteration*, qui cherchent un compromis : on n'évalue pas complètement la politique à chaque étape, mais on ne se limite pas non plus à une seule mise à jour de type *value iteration*.

## 3.5 Exemple

Considérons maintenant un petit problème très simple, qui permet de voir la propagation de la valeur sans calcul excessif.

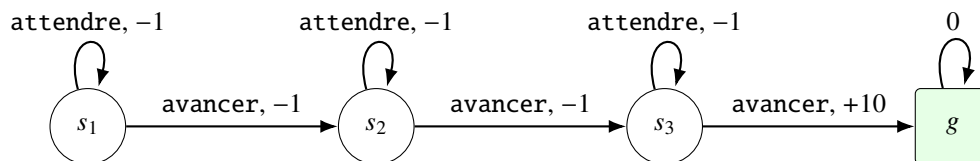


FIGURE 3.1 – Chaîne déterministe avec récompenses : l'action avancer rapproche du but, tandis que attendre coûte -1.

Les états sont  $s_1, s_2, s_3$  et un état terminal  $g$ . Dans chaque état non terminal, deux actions sont disponibles :

$$\mathcal{A} = \{\text{avancer}, \text{attendre}\}.$$

La dynamique est déterministe :

- depuis  $s_1$ , avancer mène à  $s_2$  et attendre laisse en  $s_1$  ;
- depuis  $s_2$ , avancer mène à  $s_3$  et attendre laisse en  $s_2$  ;
- depuis  $s_3$ , avancer mène à  $g$  et attendre laisse en  $s_3$  ;
- $g$  est absorbant avec récompense nulle.

Les récompenses sont :

- -1 pour attendre dans un état non terminal ;
- -1 pour avancer depuis  $s_1$  ou  $s_2$  ;
- +10 pour avancer depuis  $s_3$  vers  $g$ .

On prend  $\gamma = 0,9$ .

### Politique naïve

Considérons d'abord la politique "toujours attendre". Alors

$$v_{\pi}(s_1) = -1 + 0,9v_{\pi}(s_1),$$

$$v_{\pi}(s_2) = -1 + 0,9v_{\pi}(s_2),$$

$$v_{\pi}(s_3) = -1 + 0,9v_{\pi}(s_3),$$

d'où

$$v_{\pi}(s_1) = v_{\pi}(s_2) = v_{\pi}(s_3) = -10.$$

Cette politique est très mauvaise : elle n'atteint jamais l'état favorable.

### Propagation avec value iteration

Partons maintenant de  $v_0(s) = 0$  pour tout état. Alors

$$v_1(s_3) = \max\{-1 + 0,9 \cdot 0, 10 + 0,9 \cdot 0\} = 10,$$

$$v_1(s_2) = \max\{-1 + 0,9 \cdot 0, -1 + 0,9 \cdot 0\} = -1,$$

$$v_1(s_1) = \max\{-1 + 0,9 \cdot 0, -1 + 0,9 \cdot 0\} = -1.$$

À l'itération suivante,

$$v_2(s_2) = \max\{-1 + 0,9(-1), -1 + 0,9(10)\} = 8,$$

$$v_2(s_1) = \max\{-1 + 0,9(-1), -1 + 0,9(-1)\} = -1,9.$$

Puis,

$$v_3(s_1) = \max\{-1 + 0,9(-1,9), -1 + 0,9(8)\} = 6,2.$$

On voit très bien ce qui se passe : la forte récompense située en fin de chaîne se propage vers l'amont. D'abord  $s_3$  devient très intéressant, puis  $s_2$ , puis enfin  $s_1$ . C'est exactement le type de mécanisme que capture la programmation dynamique.

## 3.6 Transition vers le model-free RL

Les méthodes précédentes supposent que le modèle est connu. Or, dans de nombreux problèmes réels, on ne connaît pas les probabilités  $P(s' | s, a)$  ni les récompenses moyennes exactes sous forme analytique.

C'est ici qu'intervient le **model-free RL**. L'idée générale est la suivante :

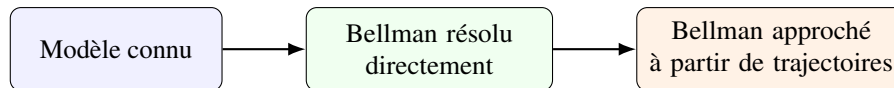
- on ne dispose pas des équations du modèle ;
- on observe seulement des trajectoires issues de l'interaction ;
- on veut néanmoins estimer des valeurs ou améliorer une politique.

Les équations de Bellman ne disparaissent pas pour autant. Elles deviennent la boussole théorique. La différence est qu'on ne peut plus les résoudre directement ; il faut les *approcher à partir d'échantillons*.

C'est ce qui mène naturellement à des méthodes comme :

- les méthodes *Monte Carlo*, qui utilisent des retours observés complets ;
- les méthodes *Temporal-Difference*, qui corrigent une estimation par une estimation future ;
- *Q-learning*, qui vise directement l'approximation de  $q_*$ .

On peut résumer le passage conceptuel ainsi :



Autrement dit, la programmation dynamique n'est pas seulement utile en soi. Elle sert aussi de fondation conceptuelle pour comprendre ce que les méthodes modernes de RL essaient d'apprendre à partir des données.

# Conclusion

Nous avons présenté le cadre général de l'apprentissage par renforcement en partant d'une description intuitive de l'interaction entre un agent et son environnement, puis en introduisant progressivement les notions d'état markovien, de politique, de retour et de fonction de valeur.

Le coeur mathématique du domaine apparaît avec les équations de Bellman. Elles expriment sous une forme récursive la relation entre récompense immédiate et intérêt du futur. À partir d'elles, les méthodes de programmation dynamique deviennent naturelles : *policy iteration* améliore explicitement une politique déjà donnée, tandis que *value iteration* vise directement la valeur optimale.

Ces deux méthodes supposent toutefois un modèle connu. La suite naturelle du cours consiste donc à supprimer cette hypothèse et à apprendre à partir de trajectoires observées. Le langage introduit ici – états, politiques, valeurs, Bellman – restera alors le même, mais les outils algorithmiques changeront. C'est précisément ce passage qui mène au *model-free reinforcement learning*.

# Références

- [1] Richard S. Sutton and Andrew G. Barto, *Reinforcement Learning : An Introduction*, 2nd edition, The MIT Press, 2018.  
<http://incompleteideas.net/book/the-book-2nd.html>
- [2] Shiyu Zhao, *Mathematical Foundations of Reinforcement Learning*, Springer, 2026.  
Projet associé : <https://github.com/MathFoundationRL/Book-Mathematical-Foundation-of-Reinforcement-Learning>
- [3] David Silver, *Lectures on Reinforcement Learning*, 2015.  
<https://www.davidsilver.uk/teaching/>